

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must be typeset (L^AT_EX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Amortized analysis of 2-3-4 trees (33 points)

Recall that in a 2-3-4 tree, whenever we insert a new key we immediately split (on our way down the tree) any node we see that is full (has 3 keys in it).

- (a) (11 points) Show that in the worst case, the number of splits that we do in a single operation can be $\Omega(\log n)$. In other words, show that there is a series of n inserts so that the next insert causes $\Omega(\log n)$ splits. You can be informal here — just explain at a high level what such a sequence would look like and why it would result in $\Omega(\log n)$ splits.

The key observation in 2-3-4 trees is that every leaf must have the same depth. Our worst case scenario is that every node along the insertion path is full (a 4-node), which means that on the way down, every node must be split. Therefore, 1 split occurs for every level of the tree. To find the lower bound for the number of splits, we must find the depth of the tree. Because each level increases the number of nodes by a constant level (between 2-4, since the number of children of any internal node is between 2 and 4), we can represent the number of levels in the tree as $\Theta(\log n)$ (as there are currently n nodes in the tree). To insert a new element, we perform 1 split at each of the $\Theta(\log n)$ levels, yielding $\Omega(\log n)$ splits.

Let's try to get around this worst-case bound by using amortized analysis. We will try to prove that the amortized number of splits is only $O(1)$. Note that this does not mean that the amortized running time of an insert is $O(1)$ (since this is not true); it just means that the amortized number of splits is $O(1)$. So think of “cost” not as “time”, but as “number of splits”. Since we didn't talk about them in class, feel free to assume there are no delete operations.

- (b) (11 points) Since we only have to split a node when it's full, a natural approach is to have a bank on every node which works as follows. When a node is first created its bank is initialized to 0. When a node becomes full, we add 1 to its bank. Then when we split a node we use the 1 in its bank to pay for the split. In other words, a node's bank is 1 if the node is full and 0 otherwise.

This argument unfortunately does not work. Explain why this banking scheme *does not* imply that the amortized number of splits is $O(1)$. Your explanation should be convincing, but does not need to be a fully formal proof

TODO

- (c) (11 points) Show how to modify this argument to work: use a bank at each node to prove that the amortized number of splits is $O(1)$. Hint: the previous part shows that this bank cannot just equal 1 if the node is full and 0 otherwise.

TODO

2 Move to Front (34 points)

Amortized analysis can be used not just to give absolute bounds on running times, but also bounds *compared to other algorithms*. In this problem we'll see an example of this.

Suppose we have n items $x_1, x_2, \dots, x_n$ that we wish to store in a linked list. The cost of a *lookup* operation is the position in the list, i.e. if we lookup some element x_i we pay 1 if it is the first element, 2 if it is the second element, 3 if it is the third element, etc. This models the time it takes to scan through the list to find the element.

For example, suppose there are four items and we lookup x_1 twice, x_2 three times, x_3 once, and x_4 four times. Then if we store them in the list (x_1, x_2, x_3, x_4) the total cost is $1(2) + 2(3) + 3(1) + 4(4) = 27$. On the other hand, if we stored them in the list (x_4, x_2, x_1, x_3) the cost would be $1(4) + 2(3) + 3(2) + 4(1) = 20$. It is easy to see that if we knew the number of times each element was looked up, the optimal list is simply sorting by the number of lookups, i.e. the first element is the one looked up most often, then the element looked up next most often, etc.

But what if we do not know how many times each element will be looked up? It turns out that a good strategy is *Move-to-Front* (MTF): when we lookup an item, we also move it to the head of the list. Let's say that moving an element is free (it is easy enough to implement with a constant number of pointer switches, in any case). So if, for example, we start with the list $(x_1, x_2, \dots, x_n)$ and then lookup x_4 , we pay a cost of 4 and afterwards the list will be $(x_4, x_1, x_2, x_3, x_5, x_6, \dots, x_n)$. Then if we lookup x_4 again we only have to pay a cost of 1.

- (a) (17 points) Consider a sequence of m operations (think of m as being much larger than n). Let C_{init} be the total cost of these operations if we used the original list $(x_1, x_2, \dots, x_n)$ the entire time (i.e., we do not use MTF). Let C_{MTF} be the total cost if we use MTF on the same operations starting from the original list. Prove that $C_{MTF} \leq 2C_{init}$.

Hint: Think of each item x_i as having a "piggy bank" in which the number of tokens is the number of elements before it in the current list that come after it in the initial list, or a potential function equal to the number of reversals.

Let us assume that for each element x_i , where i is the index of the element in the list. This means that the cost C_i of accessing that element is i . We will consider a sequence of m lookup operations, and consider the cost C of this series of operations.

Let us consider the cost of looking up some element x_i . In the initial lookup implementation, the cost would be: $C_i = i$

However, let us consider this cost of access for element x_i with the MTF lookup operation. In this, we must not only consider the initial cost but also the amount accumulated from previous swaps. For any the access of element x_i , we can consider 1 token "saved" for each element that is currently before x_i which was initially after x_i , since the process of moving x_i to the front reverses back what was already reversed.

Similar to the naive implementation, the initial lookup cost is: $C_i = i$. Since this applies to any element, we can generalize this to any lookup with the initial algorithm: $C_{init} = i$.

After a series of some operations, x_i will have changed location. We set this location to j . We can divide the set of elements before x_i after m operations into two categories: elements that were before x_i in the initial list, which we term B and elements that were originally after x_i , which we term D . Therefore, we can represent the current position j as: $j = |B| + |D| + 1$

We can represent the new cost of access of x_i as: $C_i = |B| + |D| + 1$. We know that every element D which is now before x_i started out after x_i , so there are $|D|$ tokens in the x_i 's piggy bank. We can spend these tokens to cover the cost of passing over those $|D|$ elements, but we must still pay the cost: $C_i = |B| + 1$

As we move x_i to the front, $|B|$ tokens enter x_i 's piggy bank, since they represent elements that were before and are now after x_i . However, we must still cover the cost $|B| + 1$. Because x_i was initially at position i , we know that $|B| \leq (i - 1)$. Therefore, the maximum cost of any access of x_i that is left to be paid after paying the accumulated $|D|$ tokens is: $C_i \leq i$.

In the process of accessing x_i , we must create the $|B|$ tokens, while also actually accessing the element. Thus, the operation must access the element, spend the $|D|$ tokens, and accumulate the $|B|$ tokens. We can express this as: $C_i = |B| + |D| + 1 - |D| + |B|$. This simplifies to: $C_i = 2|B| + 1$.

Inserting the equation we found for be, we get: $C_i \leq 2(i - 1) + 1 = 2i - 1 < 2i$

Therefore, this operation costs at most $2i$. This piggy bank argument establishes that the accounting cost of any MTF operation o is less than twice the cost of the comparable naive algorithm.

We can expand this over the whole sequence of m elements:

$$C_{MTF} \leq (2i_1 + 2i_2 + 2i_3 + \dots + 2i_m)$$

Now summing the initial algorithm, since the list nevver changes:

$$C_{init} = i_1 + i_2 + i_3 + \dots + i_m$$

Putting this together:

$$C_{MTF} \leq (2i_1 + 2i_2 + 2i_3 + \dots + 2i_m) = 2 * C_{init}$$

Therefore:

$$C_{MTF} \leq 2 * C_{init}$$

- (b) (17 points) Now let C_{static} be the smallest possible cost among all fixed lists. In other words, given the same sequence of lookups, each fixed list (without MTF) has some cost, and let C_{static} be the minimum of those costs. Note that this will correspond to the list that is sorted by the number of accesses, like in the first example. Prove that $C_{MTF} \leq 2C_{static} + n^2$.

Hint: Think about your bank or potential function argument from the previous part. What if you redefine it to be relative to the optimal list, rather than the initial list? Then what is the initial bank/potential value? How does this affect the amortized analysis?

In the last problem, we compared the MTF list to the original list. In this list, we define it relative to the optimal list, which is organized by number of accesses.

3 k -th smallest elements (33 points)

Given an array $a_1, a_2, \dots, a_n$ and an integer $k \in [n]$. For each $i \in [n + 1 - k]$, let b_i be the k -th smallest element in $a_1, \dots, a_{k+i-1}$.

- (a) (17 points) Design an algorithm which outputs the array $b_1, \dots, b_{n+1-k}$. The running time should be $O(n \log k)$.

I use a binary heap structure, with a maximum size of k to organize array A . We define this heap as H . The key of H is the value of each integer in A . H reverses the typical structure of the binary heap, where instead of organizing in ascending order of the key, it would organize in descending order; that is, the root of H would be the largest element, making it a max heap. From here, the first k elements would be inserted into H . Each insertion would occur at the next open leaf, and then bubble up until the correct point in the tree, wherein the element is larger than its children and less than or equal to its parents. Because during the insertions, each of the new nodes moves into the next open leaf node, where each level of the tree is filled out before adding another, the depth of the tree must be less than $(\log x) + 1$ where x is the number of nodes in the tree. Since there are k nodes in the heap, the depth of the tree would be $\log k$, meaning each insertion takes at most $O(\log k)$ time. This way, after k insertions, the k th smallest element would be the root of the heap. Thus, a function `find_max()` would be needed to return the k th smallest element without extracting it, which is simply returning the value of the root, therefore taking $O(1)$ time.

From here, the size of H would remain constant. The algorithm would iterate through each element i of H , and check whether $i \geq \text{find_max}(H)$. If so, the k th smallest element remains the same, so no further insertions into the heap must occur. If $i < \text{find_max}(H)$, then the current maximum must be extracted, and the new element i must be inserted into H , to find the new k th smallest element. The extraction process first involves swapping the rightmost leaf of H with the maximum, removing the maximum, and then the new root "swimming down" to the correct location. This involves traversing, at most, the maximum number of levels of the tree, which is $\log k$, meaning the extraction process takes at most $O(\log k)$ time. Then, i must be inserted into H , which once again, takes $O(\log k)$ time, as is proven above. Then, the new root of the tree is the new k th smallest element, so `find_max(H)` returns the next k th smallest element. Therefore, the combination of 2 $O(\log k)$ and 1 $O(1)$ operations means each insertion has an upper bound of $O(\log k)$.

For each subsequent element of A , this process repeats. Thus, for each of the n elements of A , an $O(\log k)$ operation occurs. Therefore, the overall time complexity needed to form an array of the k th smallest element of A for every subarray of A which is at least length k is $O(n \log k)$.

- (b) (16 points) Prove the correctness and the running time for your algorithm.

Correctness Proof:

We assume the correctness of the binary heap structure as per the explanation provided in the lecture slides. Therefore, in instantiating a max-heap H , we assume that the functions `insert()`, `find_max()`, and `extract_max()` exist, and they each perform their respective function described correctly. Thus, this proof will center on proving that this binary heap structure is utilized correctly in the algorithm described to solve the listed problem.

I will prove this by induction. I will first define some terms. A is the array we consider. A has n elements, and A_i is the i th element of A . Finally, I define $K(i)$ to be the condition that the k th smallest element of $A[0 : i]$ was returned by that iteration of the algorithm.

Our base case is when $A_i = A_{k-1}$. Therefore, the first k elements have been inserted into H correctly, and `find_max()` correctly returns the largest element of $A[0 : (k - 1)]$, which is by definition, the k th smallest element. Therefore, $K(k - 1)$ returns the k th smallest element of $A[0 : (k - 1)]$, so the base case holds.

For our inductive hypothesis, we assume $K(i)$ correctly finds the k th smallest element of $A[0 : i]$. We then apply algorithm K again. We will prove that this means that $K(i + 1)$ also correctly finds the k th smallest element of $A[0 : (i + 1)]$. This means that to start, we have max heap H containing the k smallest elements in $A[0 : i]$. Now, we examine element A_{i+1} .

This creates two cases:

1. $A_{i+1} \geq \text{find_max}(H)$
2. $A_{i+1} < \text{find_max}(H)$

The first case is the simplest. If $A_{i+1} \geq \text{find_max}(H)$, then the current `find_max(H)` remains the k th smallest element of $A[0 : (i + 1)]$. This is because a larger value would in no way shift which element is smaller within the relative k th smallest elements. Therefore, in this case, $K(i + 1)$ would return the same value as $K(i)$, which would remain correct, and the inductive step would hold.

We now consider the second case. In this case, there is a new smaller element than what was just the k th smallest element, so this element is now the $(k+1)$ th smallest element. Therefore, we use `extract_max(H)` to pull what was just the k th smallest element. This creates a tree with $k - 1$ nodes. Thus, to find the k th smallest element, we must insert A_{i+1} into H . Since we know `insert()` works, we can assume that after A_{i+1} is inserted into H , the root of H is now the biggest element in the heap. Because the heap now contains the k smallest elements of A , the largest of those must be the k th smallest. Thus, performing `find_max(H)` returns what is now the k th smallest element.

Therefore, we have shown that if $K(i)$ is true and the algorithm is applied again, then $K(i + 1)$ is true. In proving both the base case and the inductive step, we have proven the correctness of the algorithm.

Runtime Proof:

We will first prove that if binary heap H has k elements, it has a depth of at most $(\log k) + 1$. In my algorithm, heap H is structured so that each level of the tree must be complete before adding a node on the next level. Since each node has two children, this means that number of levels of H is strictly less than $(\log_2 k) + 1$.

We will now prove that `insert()` and `extract_max()` run in $O(\log k)$ time, and `find_max()` runs in $O(1)$ time.

Starting with the latter, we know that the maximum element of H is the root. Therefore, `find_max(H)` looks at the root of H and returns it, regardless of the number of nodes in H . Therefore, this process takes $O(1)$ time.

Let us assume we are inserting the k th element into H . Therefore, the element would slot into the next open node, making the tree now have a depth strictly less than $(\log_2 k) + 1$. Putting the node into H is one operation regardless of the number of nodes in H , so this operation is $O(1)$. However, from here, the new element must find the correct location in the H , where it is smaller than either of its children and greater than or equal to its parents. This is performed using the "swimming up" idea, where the element is swapped with its parent until the above conditions are met. Let us consider a case where the element inserted is the new maximum. This element must traverse through every level of the tree. Therefore, this process has a time complexity of $O((\log k) + 1)$, which simplifies to $O(\log k)$.

Now, we will move on to proving that removing the maximum element of heap H , with k elements, has a time complexity of $O(\log k)$. First, we must swap the root of H with the last element for simple removal. This takes constant time because `TODO`. Then, the last element is easily deleted, and the tree maintains its complete structure. From here, what was previously the last element must "sink down". First, the algorithm checks if it is smaller than either of its children. If so, it is swapped with the smaller of its children until it either has no children or it is greater than or equal to both its children. Let us consider a case where the last element of H is the smallest. Therefore, in the sinking down process, it traverses each of the $(\log k) + 1$ levels of the tree, meaning that it has a time complexity of $O((\log k) + 1)$, or $O(\log k)$.

Therefore, taken together, the process of extracting the maximum of H with k elements, inserting a new element, and finding the new maximum has 2 $O(\log k)$ and 1 $O(1)$ processes performed sequentially, meaning the time is $2 * O(\log k) + O(1)$, which simplifies to $O(\log k)$. For each of the n elements of A , some combination of these processes occur – either an `insert()` for the first k elements, a `find_max(H)` when the k smallest elements remain the same, or all 3. Thus, since this $O(\log k)$ process occurs n times, the algorithm has an overall time complexity of $O(n \log k)$.